

Finance Intelligence Platform Technical Architecture

An AI-assisted platform that automates the ingestion, validation, analysis, and reporting of financial data for Horizon Capital Partners.

CLIENT	Horizon Capital Partners
DOCUMENT	Technical Architecture Reference
PRIMARY SOURCE	<code>docs/architecture.md</code>
PREPARED	July 2026
TEST SUITE	65 automated tests passing (<code>pytest</code>)

Python 3.11

FastAPI

OpenAI LLM

n8n Workflow

Airtable

65 Tests Passing

How to read this document: every capability described here is marked either *Implemented*, *Partially implemented*, or *Design only*. Proposed/future architecture is never presented as already built — see the Component Responsibilities section for the authoritative status of each stage.

Contents

1.	System Architecture	3
2.	Component Responsibilities	5
3.	Data Flow	7
4.	Technology Choices	8
5.	Error Handling	9
6.	Testing Approach	10
7.	Scalability Considerations	11
8.	Related Documentation	12

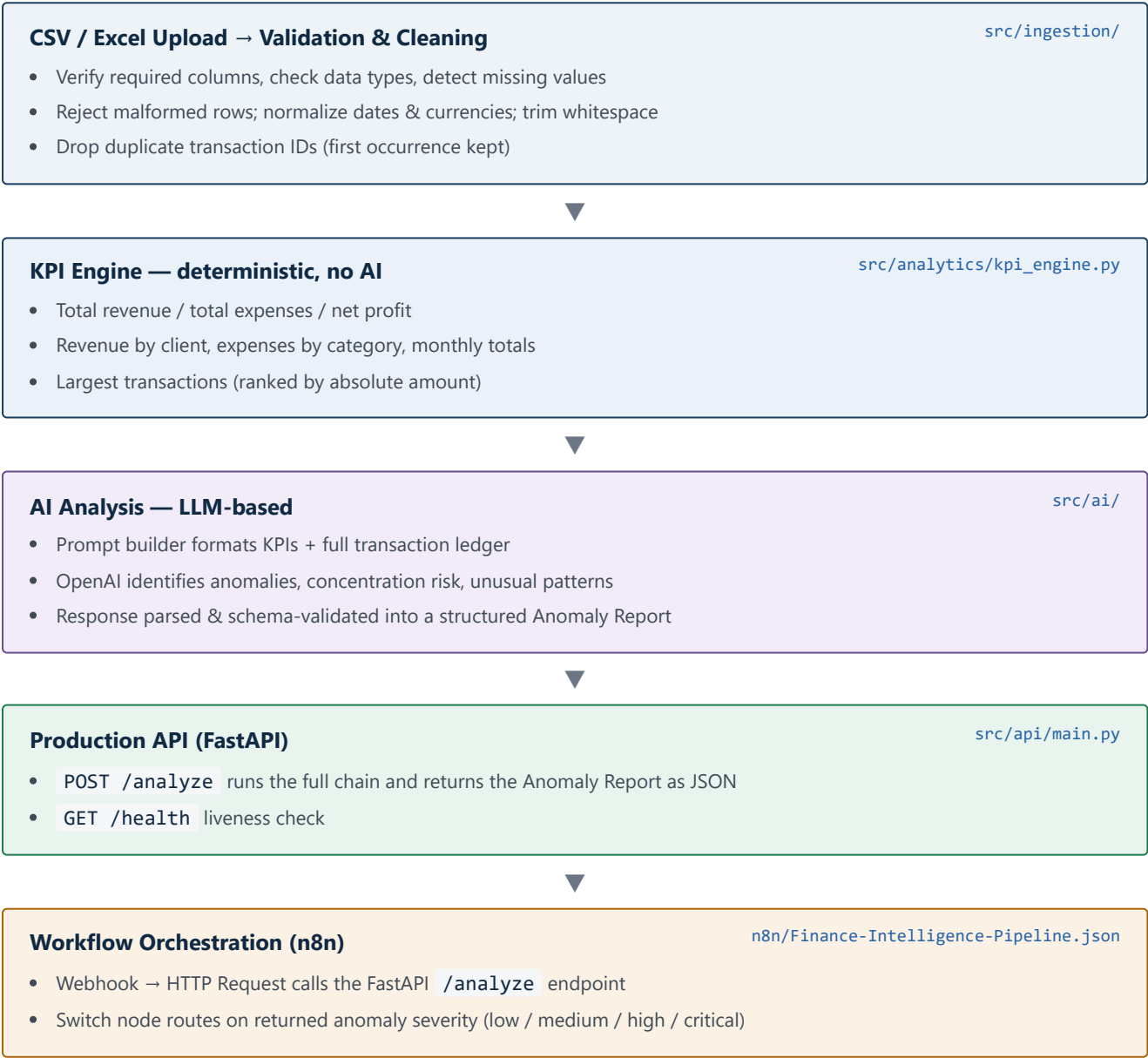
Page numbers are approximate — generated PDFs paginate based on content length at render time.

System Architecture

The Finance Intelligence Platform automates the financial reporting workflow at Horizon Capital Partners: it ingests spreadsheet data, validates and cleans it, calculates key financial metrics, uses AI to generate executive insights and flag anomalies, then stores the results and distributes reports while maintaining an audit trail.

The system is a single linear pipeline fronted by one HTTP API (FastAPI) and orchestrated end-to-end by one workflow tool (n8n). Each stage is independently testable and reviewable, so a human can verify or override any step without re-deriving the whole pipeline.

Pipeline Overview



Data Storage — Airtable

- Archive Report (per severity)
- Update Audit Trail

Notifications — Slack / Email

- Severity-routed stakeholder alerts

Not shown: a human approval gate before distribution, role-based access control, and batch processing — none of these exist yet (see Section 2).

Workflow Implementation (n8n)

The diagram above is implemented concretely as an importable n8n workflow. The screenshot below is the actual workflow graph: a **Webhook** trigger calls the FastAPI backend, a **Switch** node branches on severity, and each branch writes to Airtable and/or notifies Slack/Email; a separate failure branch archives and audits processing errors instead of silently dropping them.

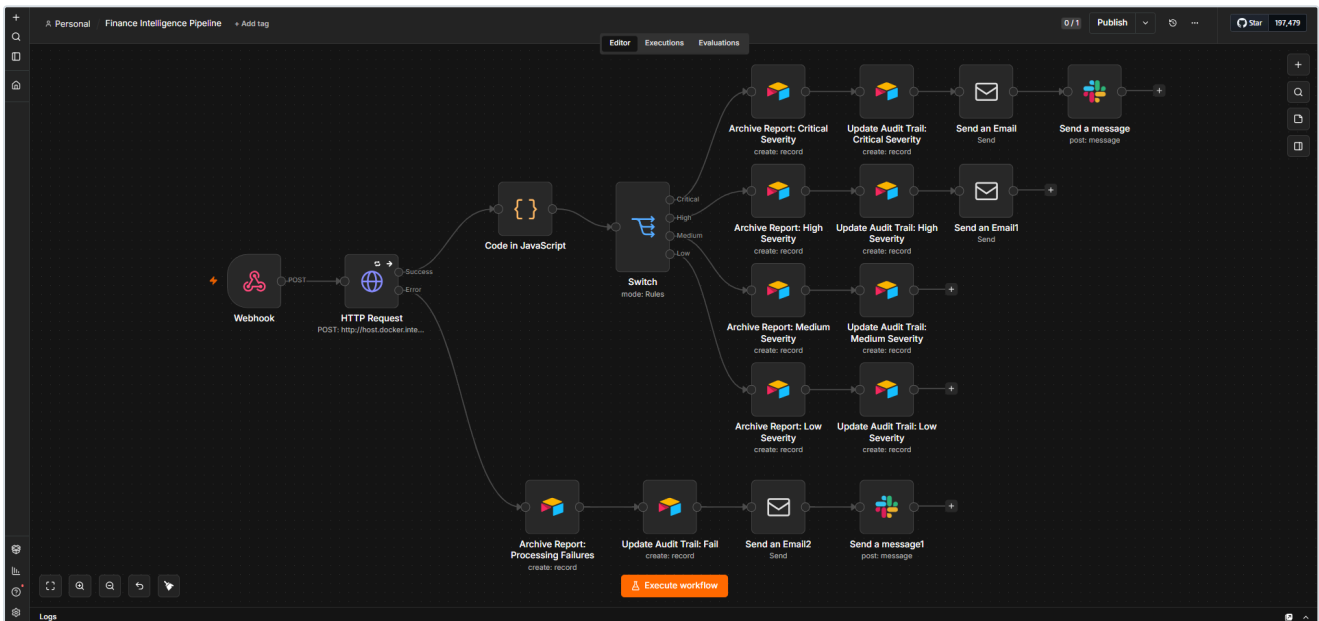


Fig. 1 — Imported n8n workflow (n8n/Finance-Intelligence-Pipeline.json), as rendered in the n8n editor.

SECTION 2

Component Responsibilities

Every stage in the pipeline maps to a concrete module. The table below states, for each stage, exactly what it owns and its current implementation status — distinguishing what is built today from what remains a design proposal.

IMPLEMENTED in production code **PARTIAL** implemented in part **DESIGN ONLY** not yet built

Stage	Owns	Status
Ingestion (CSV)	<code>src/ingestion/reader.py</code> — <code>csv.DictReader</code>	DONE
Ingestion (Excel)	<code>src/ingestion/reader.py</code> (openpyxl) — implemented but <code>/analyze</code> only accepts .csv today	PARTIAL
Validation	<code>src/ingestion/validator.py</code> — required fields, date/amount/category format, duplicate transaction ID	DONE
Cleaning	<code>src/ingestion/cleaner.py</code> — whitespace trim, currency uppercasing	DONE
Deduplication	<code>src/ingestion/deduplicator.py</code>	DONE
KPI calculation	<code>src/analytics/kpi_engine.py</code> — deterministic, unit-tested	DONE
Anomaly detection	<code>src/ai/anomaly_detector.py</code> — LLM-based; no separate statistical rule exists (see callout below)	DONE
Executive summary generation	<code>src/ai/prompt_builder.py</code> + <code>anomaly_detector.py</code>	DONE
HTTP API	<code>src/api/main.py</code> — GET <code>/health</code> , POST <code>/analyze</code>	DONE
Structured error handling	<code>src/api/errors.py</code> — no raw tracebacks returned to callers	DONE
Retries / backoff (OpenAI)	<code>src/ai/openai_client.py</code> — exponential backoff on transient failures	DONE
Logging	<code>src/logging_config.py</code> — console + rotating file (<code>logs/app.log</code>)	DONE
Workflow orchestration	<code>n8n/Finance-Intelligence-Pipeline.json</code>	DONE
Data storage (Airtable)	Per-severity Archive Report / Update Audit Trail nodes in the n8n workflow	DONE (n8n)
Notifications (Slack / Email)	Per-severity Slack/Email nodes in the n8n workflow	DONE (n8n)
Approval workflow before distribution	Design specified in <code>role-based-access-design.md</code>	NOT BUILT
Role-based access control	Design specified in <code>role-based-access-design.md</code> ; no auth in <code>src/api/</code> today	NOT BUILT

Batch processing (multi-file uploads)	Design in <code>batch-processing.md</code> ; one file per request today	NOT BUILT
Enterprise throughput (10k+ records/day)	Roadmap in <code>scalability-10k-records-per-day.md</code> ; not load-tested	NOT BUILT
Centralized monitoring / observability	File + console logs exist; no dashboard — see <code>monitoring-metrics.md</code>	PARTIAL

Anomaly detection is entirely LLM-based. There is no separate deterministic/statistical anomaly rule (e.g. a threshold or rolling-average check) sitting in front of the AI layer today. This is a known, documented trade-off — `design_review.md` recommends splitting detection into a deterministic check inside the KPI Engine feeding an LLM explanation step, but that recommendation has not been implemented. The KPI Engine itself remains deterministic-math-only; all anomaly judgment currently happens inside the AI layer.

Module Ownership Boundaries

- `src/ingestion/` — read, validate, clean, dedupe raw uploads
- `src/models/` — shared `Transaction` / `Client` data contracts
- `src/analytics/` — all financial calculation; deterministic, no AI
- `src/config.py` / `logging_config.py` — cross-cutting runtime concerns
- `src/ai/` — prompt construction, LLM invocation, response validation; never recalculates a KPI
- `n8n/` — orchestration, storage routing, notifications
- `src/api/` — HTTP orchestration only; no business logic of its own

SECTION 3

Data Flow

End-to-end, a single upload moves through the system as follows:

1. A user uploads a financial CSV to `POST /analyze` (or triggers the n8n `Webhook`, which calls the same endpoint).
2. Ingestion reads the file and validation ensures it matches the expected schema (`data_schema.md` , `validation_rules.md`).
3. Cleaning and deduplication standardize and prepare the surviving rows.
4. The KPI Engine calculates financial metrics from the validated transactions.
5. The AI layer builds a prompt from those KPIs, sends it to OpenAI, and parses the response into a structured Anomaly Report (executive summary + severity-ranked anomalies).
6. The API returns that report as JSON. When run through n8n, the Switch node routes it by severity to the `Airtable archive/audit-trail` nodes and the `Slack/Email` notification nodes; a separate failure branch handles pipeline errors instead of silently dropping them.
7. Every step logs to `logs/app.log` (API layer) and to the n8n execution history (workflow layer).

Request/Response Shape

CSV → FastAPI → Pipeline → KPI Engine → Prompt Builder → OpenAI LLM Client → OpenAI → Anomaly Detector → JSON Response

Severity Routing

Once an Anomaly Report is returned, the n8n `Switch` node inspects the highest severity present and routes to one of four branches — **low**, **medium**, **high**, or **critical** — each with its own `Airtable archive` node, `audit trail` update, and (where configured) `Slack/Email` notification. A separate failure branch (`Archive Report: Processing Failures` , `Update Audit Trail: Fail`) handles pipeline errors so they are recorded rather than dropped.

SECTION 4

Technology Choices

Component	Technology	Why
Backend API	FastAPI	Typed, fast to develop, built-in OpenAPI docs for the <code>/analyze</code> contract
Workflow orchestration	n8n	Visual workflows, built-in retries, self-hostable (ADR-001)
AI analysis	OpenAI (<code>gpt-4o-mini</code> default)	Natural-language summaries and anomaly explanations from structured KPI data
Data storage	Airtable	Simple relational storage for the MVP; not intended as the long-term store at scale
Notifications	Slack / Email (Gmail)	Stakeholder communication, routed by anomaly severity
Source data	CSV (live), Excel (library-ready)	Universal spreadsheet compatibility; <code>.xlsx</code> parsing via <code>openpyxl</code> handles formulas, merged cells, and hidden sheets

Why n8n (ADR-001)

n8n was selected over hand-rolled Python microservices or a SaaS tool such as Zapier/Make. Rationale: visual workflows are easy to review non-technically, built-in integrations reduce development time, it supports retries/error handling natively, and it is open-source and self-hostable — avoiding vendor lock-in as workflow complexity grows. The accepted trade-off is that very large workflows can become visually unwieldy; this is a documented, known cost, not an oversight.

Why OpenAI, and Where It Stops

OpenAI is used specifically for the narrative half of the AI step — turning structured KPI data into a readable executive summary and anomaly explanation. All arithmetic (totals, revenue by client, expense breakdown, monthly totals) is computed deterministically in `src/analytics/kpi_engine.py` *before* the model ever sees the data; the prompt explicitly instructs the model never to recompute or invent figures. This keeps the numbers the AI reasons over auditable, even though the anomaly *judgment* itself is LLM-based (see Section 2's callout).

Why Airtable (for now)

Airtable is an intentional MVP-scale choice: fast to stand up, no schema migrations needed to iterate, and sufficient for validated current volume. It is explicitly **not** treated as the long-term store — per-base row caps and API rate limits make it the platform's most direct constraint on reaching enterprise throughput (see Section 7).

Error Handling

API Layer

`src/api/errors.py` catches every failure mode and returns a structured `{"error": "<Type>", "message": "<detail>"}` response with an appropriate HTTP status code. Unhandled exceptions are logged with their full traceback server-side but never expose that traceback or the original exception text to the caller.

Situation	Exception Raised	HTTP Status
Upload isn't .csv	<code>InvalidFileError</code>	400
Pipeline can't read the file (e.g. bad encoding)	<code>CSVProcessingError</code>	400
Every row fails validation	<code>ValidationFailedError</code>	422
LLM response isn't valid / schema-matching JSON	<code>AnomalyGenerationError</code>	502
OpenAI request fails after retries exhausted	<code>OpenAIServiceError</code>	502
OpenAI request times out on every attempt	<code>OpenAITimeoutServiceError</code>	504
Missing <code>OPENAI_API_KEY</code>	<code>ConfigurationError</code>	500
Anything else unhandled	generic handler	500

OpenAI Client Resilience

`src/ai/openai_client.py` retries only genuinely transient failures — connection errors, timeouts, rate limits, and 5xx responses — with exponential backoff (1s, 2s, 4s, ..., scaled by `Settings.openai_retry_count`). Non-retryable errors (bad request, auth failure, malformed/empty response) raise immediately, since retrying them cannot succeed. All four custom exceptions (`OpenAITimeoutError`, `OpenAIRequestError`, `OpenAIResponseError`, `OpenAIConfigurationError`) subclass a common `OpenAIClientError`, so callers can catch one type when only "did the integration fail" matters.

Workflow-Level Handling (n8n)

A dedicated failure branch in the n8n workflow archives and audits processing errors (`Archive Report: Processing Failures`, `Update Audit Trail: Fail`) rather than dropping them. Notifications are only triggered from the success and failure branches that already ran — there is no separate silent-drop path in the current workflow.

Design principle: no failure mode in this system returns a raw Python traceback or unredacted exception text to a caller. Every error path is a typed, intentional response.

Testing Approach

The automated suite covers every module in `src/` end to end — ingestion, KPI calculation, prompt building, AI response parsing, the OpenAI client's retry/backoff behavior, and the FastAPI endpoints — with **no test making a real network call**. LLM calls are faked (a `RecordingLLMClient` / `FakeLLMClient`) or mocked (`unittest.mock.patch` on the OpenAI SDK) throughout.

```
python -m pytest -q → 65 passed
```

Test File	Feature Covered	Tests
<code>test_ingestion.py</code>	CSV/XLSX reading, validation, cleaning, deduplication, hidden sheets, merged cells	9
<code>test_kpi_engine.py</code>	Revenue/expense/net-profit totals, per-client and per-category grouping, monthly totals, largest transactions	6
<code>test_prompt_builder.py</code>	Prompt section structure, JSON schema documentation, ledger truncation, currency formatting	12
<code>test_anomaly_detector.py</code>	LLM response parsing/validation, malformed JSON handling, severity normalization, timezone-aware timestamps	18
<code>test_openai_client.py</code>	Retry/backoff, exhausted retries, non-retryable errors, empty/malformed responses, exception hierarchy	11
<code>test_api.py</code>	HTTP endpoints, error-to-status-code mapping, scrubbed 500 responses	9

Manual & End-to-End Scenarios

Beyond the automated suite, `docs/test-scenarios.md` records manual scenarios exercised through the FastAPI Swagger UI and the full imported n8n workflow: CSV upload, XLSX rejection (API currently accepts CSV only), duplicate transactions, missing required values, currency/whitespace normalization, an OpenAI timeout retrying three times before routing to the failure workflow, the health endpoint, and empty/malformed CSV handling. All ten recorded scenarios pass.

Testing Principles Applied

- **No network calls in the automated suite** — every OpenAI interaction is faked or mocked, so the suite runs deterministically and offline.
- **Behavior over text-pinning** — prompt-builder tests assert on section headings, labels, and counts, not the full rendered prompt string, so cosmetic copy changes don't break tests.
- **Real fixtures for real edge cases** — XLSX tests build actual workbooks in-memory (hidden sheets, merged cells) with `openpyxl` rather than hand-crafted mocks.
- **Error paths are first-class** — malformed JSON, missing fields, invalid severities, exhausted retries, and unhandled exceptions are all explicitly tested, not just the happy path.

Scalability Considerations

The current architecture is a single FastAPI instance, a synchronous pipeline, and one n8n workflow processing one file at a time. This is the right foundation to validate correctness first — it has **not** been load-tested against, or proven capable of, an enterprise-scale volume (10,000+ records/day). The recommendations below are a migration path that extends the existing pipeline; none require a rewrite.

Current Constraints

- Synchronous request handling — `/analyze` blocks for the full pipeline duration, including the OpenAI round-trip
- Single upload per request — no batch endpoint; N files require N sequential requests
- No distributed workers or documented multi-instance deployment
- Airtable serves as both report store and audit trail, sharing one rate-limit budget for both
- One OpenAI request per upload — no batching across files
- Entire file held in memory during ingestion — unproven beyond sample-data row counts
- No caching layer — every upload is fully reprocessed, even if identical to a prior one
- No concurrency/backpressure handling for simultaneous uploads

Enterprise Scalability Roadmap (proposed — not implemented)

Layer	Recommendation
Compute	Horizontal FastAPI scaling behind a load balancer; convert <code>/analyze</code> to <code>async def</code> and move blocking I/O off the request thread
Processing model	Queue-based architecture (e.g. Celery/RQ or n8n queue mode) decoupling "file received" from "file processed"; dedicated worker pools, with AI workers scaled independently since OpenAI is the slowest step
Data storage	Migrate off Airtable to a transactional store (e.g. PostgreSQL) with real concurrency guarantees and no per-base rate ceiling; add indexing and partitioning by client/date
Caching	Redis (or equivalent) for OpenAI response caching keyed by prompt hash, and KPI result caching for repeated uploads
Infrastructure	Containerization, orchestration (Kubernetes/Docker Swarm), autoscaling, cloud object storage for raw file archival
AI cost/latency	Request queuing for OpenAI calls, response caching, prompt-size minimization beyond the current 200-row ledger cap, token-usage monitoring
Observability	Aggregate metrics (requests/min, queue depth, OpenAI latency, Airtable error rate, throughput vs. the 10k/day target) into a real dashboard, replacing today's per-request log lines

Framing: every item above is additive to the existing `Read → Validate → Clean → Dedupe → KPI Engine → AI → Store → Notify` pipeline. Reaching enterprise-scale throughput means changing *how much of the pipeline runs concurrently and where its data lives*, not changing what the pipeline does.

Related Documentation

This document summarizes `docs/architecture.md` for a technical audience. For full detail, see the source documents in the project repository:

Document	Covers
<code>docs/architecture.md</code>	Primary source for this document — full pipeline, component status, data flow, error handling, security
<code>docs/decisions.md</code>	Architecture Decision Records (ADR-001: n8n as orchestration platform)
<code>docs/data_schema.md</code> / <code>validation_rules.md</code>	The ingestion contract — column definitions and validation rules
<code>docs/role-based-access-design.md</code>	RBAC and approval-workflow design (not implemented)
<code>docs/batch-processing.md</code>	Batch-upload design (not implemented)
<code>docs/scalability-10k-records-per-day.md</code>	Full scaling roadmap and gap analysis
<code>docs/performance-optimization.md</code>	Implemented optimizations vs. enterprise-scale recommendations
<code>docs/monitoring-metrics.md</code>	Current logging vs. proposed centralized observability
<code>docs/design_review.md</code>	Pre-build architecture review — open risks and recommendations
<code>docs/automated-test-coverage.md</code> / <code>test-scenarios.md</code>	Per-test breakdown and manual end-to-end scenarios
<code>docs/Explainer.md</code>	Chronological build log — every file created/modified, and why

Generated from `docs/architecture.md` and the current state of the `src/`, `tests/`, and `n8n/` directories. Where this document and the Markdown source diverge in the future, `docs/architecture.md` remains authoritative.